

Data Quality Fundamentals - Overview

In this train, we'll discover how to ensure your data is clean and reliable through data quality validation and testing. We'll take a look at the fundamentals of safeguarding data integrity and get acquainted with essential tools for maintaining high data quality standards.

Learning Objectives

By the end of this train, you should be able to:

- Understand the need for data quality validation and testing, including:
 - Data quality validation as an assurance of incoming data.
 - Data testing as integration testing.
- Define the six dimensions of data quality.

Why Data Quality Matters

Data is only as valuable as it is *reliable*. Poor-quality data can lead to flawed business decisions, broken models, and misleading analyses. That's why it's critical to implement validation and testing throughout your data pipelines — not just at the end.

Data quality validation focuses on checking the integrity of data as it enters your system — is it complete, accurate, and in the expected format?

Data testing plays a role similar to software integration testing. It helps ensure that transformations, joins, aggregations, and other operations produce consistent and correct results within the data pipeline.

In most real-world scenarios, raw data from source systems is rarely perfect. It often arrives with missing values, inconsistent formats, and hidden errors. Data engineers spend a significant amount of time on data cleaning and transformation to fix these issues — which is precisely why formal data quality frameworks and tools are so essential.

The Six Dimensions of Data Quality

Data quality can be assessed using the following six core dimensions:

- **Accuracy:** Does the data correctly represent real-world values?
- **Completeness:** Are any required values missing?
- **Consistency:** Does the data remain uniform across different datasets and systems?
- **Timeliness:** Is the data available when it's needed? Is it up to date?
- **Uniqueness:** Are there duplicate records where there shouldn't be?
- **Validity:** Does the data conform to the expected format, types, or rules?

Impact of Poor Quality: Practical Examples

Scenario 1 — Revenue Report: A multinational corporation aggregates sales from European (EUR) and North American (USD) systems.

- **Problem (Uniqueness):** A bug causes several high-value European transactions to be loaded twice, artificially inflating revenue.
- **Problem (Consistency):** One system uses USD, another uses US-DOLLAR. The aggregation logic fails to recognise US-DOLLAR, excluding millions in sales.

Result: A completely unreliable report that erodes investor confidence and may trigger regulatory fines.

Scenario 2 — Birthday Campaign: An e-commerce company runs a "Happy Birthday" email campaign using CRM data.

- **Problem (Completeness):** 40% of customer records are missing a `date_of_birth` — those customers can't be included.
- **Problem (Timeliness):** The CRM sync only runs once a week, causing many customers to receive birthday emails several days late.

Result: Lost revenue opportunities and damaged customer perception.

Consider this:

Imagine you are designing a data system for a healthcare provider. Every day, data comes in from hundreds of hospitals. If the data is complete, consistent, current, and accurate — how would this ideal state influence decision-making and patient care?

A Glimpse into Implementation

Before using advanced tools, you can think of data quality checks as simple functions. Here's a basic Python example checking two dimensions for a single record:

```
import re

def check_patient_data(record):
    # 1. Check for Completeness
    if record.get('email') is None:
        print(f"Failed Completeness: Email missing for ID {record.get('id')}")
        return False

    # 2. Check for Validity
    email_regex = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if not re.match(email_regex, record['email']):
        print(f"Failed Validity: Email '{record['email']}' has an invalid format.")
        return False

    print(f"Record ID {record.get('id')} passed checks.")
    return True
```

```
# Sample Data
patient_1 = {'id': 101, 'email': 'user@example.com'}
patient_2 = {'id': 102, 'email': None}
patient_3 = {'id': 103, 'email': 'invalid-email'}

# Run Checks
check_patient_data(patient_1)
check_patient_data(patient_2)
check_patient_data(patient_3)
```

Looking Ahead

In the upcoming modules, we'll dive deeper into:

- How to implement data quality checks using tools like **Great Expectations** and **PyDeequ**.
- Best practices for testing data transformations, joins, and aggregates.
- Design patterns for scalable and automated data validation pipelines.

Conclusion

Mastering the fundamentals of data quality is a critical first step for any data professional. By understanding the six core dimensions and the distinct roles of validation and testing, you can build a framework to ensure your data is reliable, accurate, and trustworthy.

Key Takeaways



- **Takeaway 1:** Data Quality Validation: Ensures incoming data meets expectations.
- **Takeaway 2:** Data Testing: Verifies that data behaves as expected within a system or pipeline.
- **Takeaway 3:** Six Key Dimensions: Accuracy, Completeness, Consistency, Timeliness, Uniqueness, and Validity.
- **Takeaway 4:** The Reality: Raw data is rarely perfect and requires significant cleaning effort by data engineers.
- **Takeaway 5:** Business Impact: Poor data quality directly leads to flawed decisions and negative business outcomes.